```
module sobel_edge_detection #(parameter WIDTH = 8) (
input wire clk, // Clock signal
input wire rst, // Reset signal
input wire [WIDTH-1:0] pixel_in, // Input pixel (current pixel)
input wire [WIDTH-1:0] pixel_in_top, // Pixel above the current pixel
input wire [WIDTH-1:0] pixel_in_bottom, // Pixel below the current pixel
input wire [WIDTH-1:0] pixel_in_left, // Pixel to the left
input wire [WIDTH-1:0] pixel_in_right, // Pixel to the right
input wire [WIDTH-1:0] pixel_in_tl, // Top-left corner pixel
input wire [WIDTH-1:0] pixel_in_tr, // Top-right corner pixel
input wire [WIDTH-1:0] pixel_in_bl, // Bottom-left corner pixel
input wire [WIDTH-1:0] pixel_in_br, // Bottom-right corner pixel
output reg [WIDTH-1:0] edge_out, // Output edge strength
output reg ready // Ready signal indicating output is valid
);
// Sobel kernels for X and Y direction
// Sobel X kernel
reg signed [WIDTH-1:0] sobel_x [0:2][0:2]; // 3x3 kernel for X direction
// Sobel Y kernel
reg signed [WIDTH-1:0] sobel_y [0:2][0:2]; // 3x3 kernel for Y direction
// Temporary variables to hold the gradient sums
reg signed [15:0] gradient_x;
reg signed [15:0] gradient_y;
reg signed [15:0] gradient_magnitude;
// Process the input on each clock cycle
always @(posedge clk or posedge rst) begin
if (rst) begin // Initialize Sobel kernels
sobel_x[0][0] = -1; sobel_x[0][1] = 0; sobel_x[0][2] = 1; //Sobel X kernel for detecting horizontal edges
sobel_x[1][0] = -2; sobel_x[1][1] = 0; sobel_x[1][2] = 2;
sobel_x[2][0] = -1; sobel_x[2][1] = 0; sobel_x[2][2] = 1;
sobel_y[0][0] = -1; sobel_y[0][1] = -2; sobel_y[0][2] = -1; // Sobel Y kernel for detecting vertical edges
```

```
sobel_y[1][0] = 0; sobel_y[1][1] = 0; sobel_y[1][2] = 0;

sobel_y[2][0] = 1; sobel_y[2][1] = 2; sobel_y[2][2] = 1;

gradient_x <= 0;

gradient_y <= 0;

gradient_magnitude <= 0;

edge_out <= 0;

ready <= 0;

end

else begin

// Calculate the gradient in X direction

gradient_x <= (sobel_x[0][0] * pixel_in_tl) + (sobel_x[0][1] * pixel_in_top) + (sobel_x[0][2] *
pixel_in_tr) +

(sobel_x[1][0] * pixel_in_left) + (sobel_x[1][1] * pixel_in) + (sobel_x[1][2] * pixel_in_right) +

(sobel_x[2][0] * pixel_in_bl) + (sobel_x[2][1] * pixel_in_bottom) + (sobel_x[2][2] * pixel_in_br);

// Calculate the gradient in Y direction

gradient_y <= (sobel_y[0][0] * pixel_in_tl) + (sobel_y[0][1] * pixel_in_top) + (sobel_y[0][2] *
pixel_in_tr) +

(sobel_y[1][0] * pixel_in_left) + (sobel_y[1][1] * pixel_in) + (sobel_y[1][2] * pixel_in_right) +

(sobel_y[2][0] * pixel_in_bl) + (sobel_y[2][1] * pixel_in_bottom) + (sobel_y[2][2] * pixel_in_br);

// Calculate the gradient magnitude (simplified: no square root)

gradient_magnitude <= (gradient_x * gradient_x) + (gradient_y * gradient_y);

edge_out <= (gradient_magnitude[15:8] > 8'd255) ? 8'd255 : gradient_magnitude[15:8]; // Saturate
if > 255

// Signal that the result is ready

ready <= 1;

end

end

endmodule
```